A Project Report

On

“SMART IRRIGATION SYSTEM USING IOT AND CLOUD”

Submitted in partial fulfillment of the requirement for the award of the degree ofBACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

By:

PAWAN KUMAR MAURYA (100210017)

HIMANSHU CHAUDHARY (100210014)

YASH KUMAR (100210019)

UNDER THE GUIDANCE OF

Ms. Priyanka Singh

(Assistant Professor, ECE)
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

NAAC A++ Accredited

SIR CHHOTU RAM INSTITUTE OF ENGINEERING & TECHNOLOGY

CHAUDHARY CHARAN SINGH UNIVERSITY, MEERUT

(2025)

SIR CHHOTU RAM INSTITUTE OF ENGINEERING & TECHNOLOGY

CHAUDHARY CHARAN SINGH UNIVERSITY MEERUT

Approved by A.I.C.T.E., New Delhi

NAAC A++ Accredited

Student Declaration/ Certificate-I

This is to certify that the project titled **"Smart irrigation system Using IOT and Cloud "** in Meerut is submitted in partial fulfillment of the requirement of the degree of Bachelor of Technology in Electronics & Communication engineering of **Sir Chhotu Ram Institute of Engineering and Technology, Chaudhary Charan Singh University Campus, Meerut (U.P)** under the supervision.

Er. Priyanka Singh
(Project Supervisor)

Dr. Pankaj Kumar
(Project Coordinator)

SIR CHHOTU RAM INSTITUTE OF ENGINEERING & TECHNOLOGY
CHAUDHARY CHARAN SINGH UNIVERSITY MEERUT
Approved by A.I.C.T.E., New Delhi

NAAC A++ Accredited

Student Declaration/ Certificate-II

The project report titled **"Smart irrigation system Using IOT and Cloud"** in Meerut is submitted by **Pawan Kumar Maurya (100210017), Yash Kumar (100210019) and Himanshu Chaudhary (100210014)**. It warrants its acceptance as a prerequisite for the degree of **Bachelor of Technology in Electronics and Communication engineering**.

Er. Priyanka Sing
(Internal Examiner)

…...…….
(External Examiner)

Dr. Pankaj Kumar
(Coordinator)
Dept. Of Electronics & Communication Engineering

Dr. Niraj Singhal
(Director, SCRIET)

ACKNOWLEDGEMENT

We extend our thanks to **Dr. Niraj Singhal, Director** and **Coordinator, Dr. Pankaj Kumar**, for giving us the motivation and guidance to complete this project.

We are deeply indebted to our project guide, **Ms. Priyanka Singh,** for the initial idea of the project and for all the guidance and encouragement he gave in the subsequent months. His supervision and co-operation at every stage helped us in successfully completing the project. Whatever intellectual effort may be reflected from this project is the direct result of the informative and stimulating discussions and suggestions he has given during the course of the project.

Lastly, we would like to thank the entire staff of the **Electronics & Communication Engineering** and our college for their support and cooperation in the completion of this project.

Pawan Kumar Maurya (100210017)

Yash Kumar (100210019)

Himanshu Chaudhary (100210014)

Table of Contents

ABSTRACT

The Smart Irrigation System with Cloud is an advanced solution designed to optimize water usage in agricultural fields by leveraging IoT (Internet of Things) technology and cloud computing. This system incorporates a network of soil moisture sensors, weather forecasting data, and automated irrigation controllers to monitor and manage the irrigation process in real-time. The cloud platform serves as the central hub for data collection, processing, and analysis, providing farmers with valuable insights into soil conditions, water requirements, and irrigation efficiency.

By using data from the soil sensors and external weather conditions, the system determines the optimal amount of water required for irrigation, reducing water wastage and ensuring the healthy growth of crops. Through a mobile or web interface, farmers can access real-time information about their fields, track water consumption, and receive notifications on irrigation schedules. Furthermore, the cloud-based system offers scalability, remote control, and predictive analytics, allowing farmers to make informed decisions and improve overall water management practices.

This Smart Irrigation System not only enhances water conservation but also contributes to sustainable farming practices by reducing labor costs, improving crop yields, and minimizing environmental impact

Chapter: 1

Introduction

Water is one of the most valuable resources in agriculture, and its efficient management has become increasingly important due to climate change, rising population, and limited water availability. Traditional irrigation methods often lead to overwatering or underwatering, resulting in poor crop yield and resource wastage. To tackle this challenge, modern farming needs a smart, automated, and remote-controllable system.

This project presents a Smart Irrigation System based on the ESP32 microcontroller, designed to optimize water usage using real-time environmental data. The ESP32 reads inputs from a soil moisture sensor and a DHT11 temperature-humidity sensor, and accordingly controls a water pump via a relay. If soil moisture drops below a set level, the system turns the pump on automatically — and off again once adequate moisture is restored.

What sets this system apart is its Telegram Bot integration, allowing users to control the pump remotely, receive sensor data, schedule reports, and even change Wi-Fi credentials on the fly. It also logs data to ThingSpeak for real-time monitoring and Google Sheets for historical tracking — ensuring both transparency and accessibility.

With features like secure user authentication, remote Wi-Fi reconfiguration, scheduled reporting, and Telegram -based pump alerts, this system is tailored to be user-friendly for both tech-savvy and non-technical users.

This smart irrigation solution is suitable for agricultural fields, home gardens, greenhouses, and urban landscaping, helping save water, reduce manual labor, and increase crop productivity through intelligent automation.

Chapter : 2

Features of project

2.1 Features of Smart irrigation system using IOT & Cloud :

Smart Irrigation System is designed to provide an efficient, automated, and user-friendly solution for managing irrigation in agricultural, residential, and urban environments. By integrating sensors, wireless communication, and cloud-based control, this system intelligently monitors soil and environmental conditions to control watering operations with precision.In this project include many feature which is given below

1. Sensor And Monitoring Feature

- Real-time Soil Moisture Monitoring
- Accurate Temperature Reading
- Humidity Monitoring
- Change In Pump State – Only logs if sensor data changes
- Time-Stamped Logging with IST conversion

2. Pump Control And Automation

- Automatic Pump On/Off – Based on moisture threshold value
- Manual Pump Control via Telegram Bot
- Pump Status Request
- Instant Bot Notification when pump turns ON/OFF

3. Telegram Bot Integration

- Secure Access – Only accepts commands from authorized chat Person
- Smart Command Menu (`/start`) – Lists all available commands
- Wi-Fi Change By Telegram Bot – `/New_WiFi_Add` starts setup

4. Cloud & Data Logging

- ThingSpeak Integration – Live charting of sensor data
- Google Sheets Logging – Each changes event store in google sheet
- In-Memory Event Storage (200 entries) – Local buffer for reporting

5. Reports & Scheduling

- ➢ Scheduled Report Every 12 Hours
- ➢ Daily Summary Report
- ➢ Monthly Summary Report
- ➢ Last 10 Event Summary
- ➢ Auto-upload to ThingSpeak and Google Sheet

6. Project Identity

- ➢ Developer Info Command – Displays contributor names and college details
- ➢ Submission Info – Designed for academic ECE project (SCRIET CCSU)

Chapter : 3

Component used and their Description

3.1. Introduction to ESP32

The **ESP32** is a low-cost, low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities, developed by **Espressif Systems**. It has become one of the most popular and versatile boards for embedded systems and IoT (Internet of Things) applications, offering a wide range of features that make it suitable for a variety of projects—from hobbyist applications to commercial solutions.

3.1.1. Overview of the ESP32

At the heart of the ESP32 is a **dual-core processor** based on the **Xtensa LX6 architecture**. This gives the ESP32 a significant edge in performance over its predecessor, the **ESP8266**, and makes it ideal for more demanding applications. The ESP32 operates at clock speeds of up to **240 MHz**, offering excellent processing power while maintaining efficient energy consumption.

The board is equipped with built-in **Wi-Fi** (802.11 b/g/n) and **Bluetooth** (both classic Bluetooth and Bluetooth Low Energy or BLE), making it highly capable for wireless communication. This connectivity is crucial for modern IoT projects, where devices need to communicate over the internet or directly with each other.

Key Features of the ESP32 Board

Dual-Core Processing Power: The ESP32 features two **32-bit cores** that can work independently or together, which allows the board to perform multiple tasks simultaneously. This makes it suitable for real-time applications and helps in handling tasks like running a web server while also controlling sensors or motors.

Wi-Fi and Bluetooth:

- **Wi-Fi** support is built into the ESP32, providing seamless integration into wireless networks.
- **Bluetooth** functionality supports both classic Bluetooth (for devices like headphones or speakers) and Bluetooth Low Energy (BLE), making it ideal for applications requiring low-power wireless communication, such as wearable devices and home automation systems.

Rich I/O Options: The ESP32 comes with a wealth of GPIO (General Purpose Input/Output) pins, supporting various interfaces like SPI, I2C, UART, PWM, ADC, and DAC. This gives users the ability to connect a wide variety of sensors, actuators, displays, and other peripherals, making it flexible for different project needs.

Low Power Consumption: One of the key selling points of the ESP32 is its low power consumption. It has several power modes, including deep sleep mode, where it can consume as little as 10 μA of current. This makes it perfect for battery-operated devices like weather stations or remote sensing systems.

Memory: The ESP32 is equipped with 512KB of SRAM and has the ability to support external flash storage of up to 16MB (depending on the variant), allowing for plenty of space to store firmware and user data. The board also supports features like SPI Flash, which is commonly used to store the program code and other necessary resources.

Integrated Analog and Digital Peripherals: The board comes with integrated ADC (Analog-to-Digital Converter) and DAC (Digital-to-Analog Converter), making it suitable for interfacing with analog sensors and actuators. The ADC supports up to 12-bit resolution, allowing for precise readings from analog devices.

Real-Time Clock (RTC): The ESP32 has a built-in RTC (Real-Time Clock) module that can help manage time-sensitive operations. This is particularly useful for applications where time synchronization is crucial, such as data logging systems or alarms.

Security Features: The ESP32 is designed with a focus on security. It supports hardware encryption for securing data, such as SSL/TLS protocols for secure communication over Wi-Fi. The board also features secure boot, flash encryption, and RSA (Rivest-Shamir-Adleman) key generation, which helps protect the device from tampering and unauthorized access.

3.1.3. ESP32 Development and Programming

The ESP32 is an open-source hardware platform, which has contributed to its widespread adoption. It can be programmed in a variety of environments, but the most common ones are:

Arduino IDE: The **Arduino IDE** offers a user-friendly development environment and allows developers to write code using a simple programming language. With the addition of the **ESP32 core for Arduino**, users can easily develop applications for the ESP32 just like they would for other Arduino-compatible boards. The Arduino community also provides many libraries and examples to help get started quickly.

3.1.4. Pinout of ESP32 Board

I will make a separate dedicated tutorial on ESP32 Pinout. But for the time being, take a look at the pinout diagram of the ESP32 Development Board. 6/7 This pinout is for the 30 – pin version of the ESP Board. In the pinout tutorial, I will explain the pin out of both the 30 – pin as well as the 36 – pin version of the ESP Boards

Figure 3.1 : ESP 32 Pin Diagram

3.2. Soil Moisture sensor

A soil moisture sensor is a device designed to measure the water content in the soil, providing essential data for monitoring soil conditions and managing irrigation systems. These sensors are commonly used in agriculture, gardening, and environmental monitoring to ensure optimal soil moisture levels for plant health, improve water conservation, and enhance crop yields.

Overview of Soil Moisture Sensors

Soil moisture sensors detect the amount of water
present in the soil by measuring the electrical properties of the soil, as water has a high dielectric constant, affecting the electrical resistance or capacitance. By assessing these properties, the sensor can determine whether the soil is too dry, too wet, or at optimal moisture levels for plant growth

Pinout of Soil Moisture

The soil moisture sensor is extremely simple to use and only requires four pins to connect.

A0 (Analog Output) : generates analog output voltage proportional to the soil moisture level, so a higher level results in a higher voltage and a lower level results in

a lower voltage.

DO (Digital Output) : indicates whether the soil moisture level is within the limit. D0 becomes LOW when the moisture level exceeds the threshold value (as set by the potentiometer), and HIGH otherwise.

VCC : supplies power to the sensor. It is recommended that the sensor be powered from 3.3V to 5V. Please keep in mind that the analog output will vary depending on the voltage supplied to the sensor.

GND is the ground pin.

Figure 3.2: Moisture sensor

4.3. Humidity & Temperature sensor

In this project we have used a DHT 11 Humidity sensor . The DHT11 is a widely used digital humidity and temperature sensor. It is affordable, easy to use, and commonly found in hobbyist and DIY electronics projects, especially in environments that need basic humidity and temperature measurements.

Key Features:

Humidity Measurement: The DHT11 can measure humidity in the range of **20% to 80%**, with an accuracy of ±5% RH (Relative Humidity).
Temperature Measurement: It can also measure temperature in the range of **0°C to 50°C** (32°F to 122°F), with an accuracy of ±2°C.
Digital Output: The sensor provides a digital signal, making it easier to interface with microcontrollers like **Arduino**, **Raspberry Pi**, or other embedded systems.
Low Power Consumption: The DHT11 is low power and operates on a voltage of 3.5V to 5.5V, making it suitable for battery-powered projects.

Figure 3.3: DHT11 Sensor

4.4. Water Pump motor

This pump is useful for watering in the plant .It operates on 5v to 6v Voltage rating with high RPM . This motor's wire is connected to terminal block with screw

Figure 3.4: water pump

4.6. Other Components

- 2 pin Straight PCB screw terminal Block connector
- Water pump motor
- Power supply
- water pipe
- Small water tank
- Pot for plant
- Connecting wires
- Switch
- Zero PCB (Printed circuit board)
- npn transistor

Chapter : 4

Circuit Diagram And Working

4.1 Circuit Diagram and working

The Smart Irrigation System Using IOT & Cloud begins functioning the moment it is powered on via a USB adapter or power bank. Upon startup, it initializes all its connected components, including the soil moisture sensor, the DHT11 temperature and humidity sensor, and prepares its Wi-Fi and Telegram communication modules. Once active, the system continuously monitors the soil moisture level by reading analog signals from the soil sensor, and also checks the surrounding temperature and humidity via the DHT11 sensor. This environmental data is used to make intelligent irrigation decisions.

If the soil is detected as dry—based on a pre-set threshold—the ESP32 triggers a connected relay module that activates the water pump, supplying water to the soil. Once the moisture level is restored and reaches the desired level, the system automatically switches the pump off. This ensures that water is only used when needed, preventing both overwatering and under-watering.

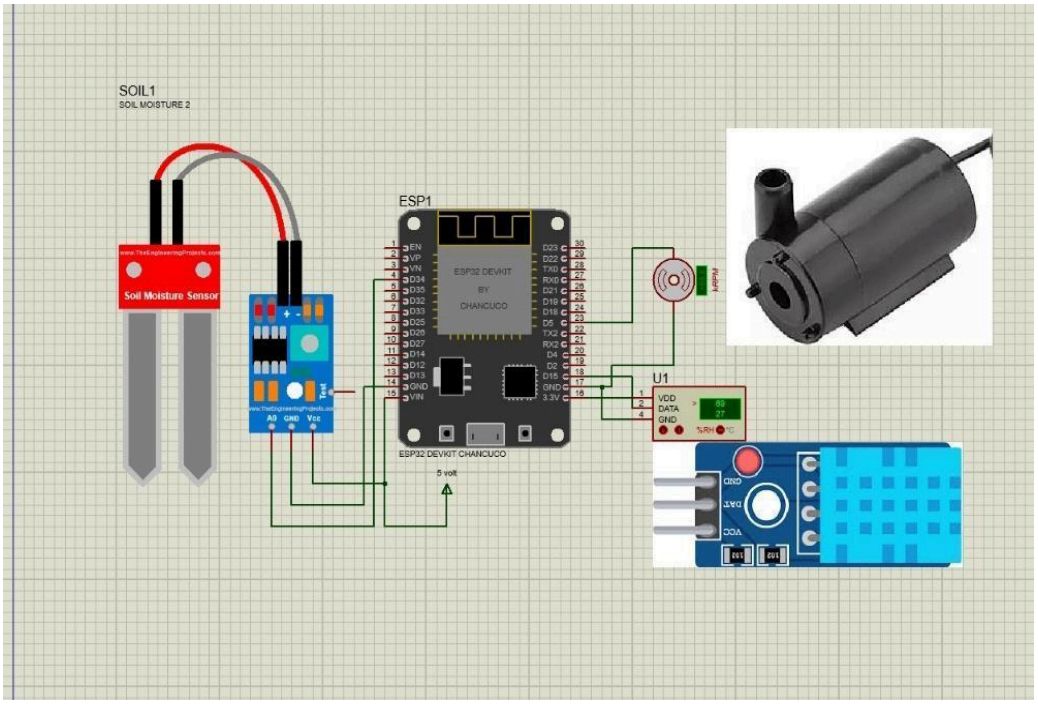


Figure 4.1. Schematics of Smart Irrigation System Using IOT & Cloud

The system also allows the user to manually control the pump or retrieve sensor data using a secure Telegram bot. Authorized users can send simple commands like /PUMP_ON, /PUMP_OFF, or /CURRENT_SENSOR_DATA to interact with the system in real-time. Additionally, all sensor readings and pump events are logged to cloud platforms like ThingSpeak (for visualization) and Google Sheets (for historical analysis).

The system stores up to 200 local event records and automatically sends detailed reports to the user every 12 hours via Telegram.

If the device is reset or loses power, it reverts to its default Wi-Fi mode, allowing users to reconnect and reconfigure the system quickly using the Telegram bot. With secure access control, real-time decision-making, and full remote functionality, the system mimics the way a smart human assistant would manage irrigation—reliably, efficiently, and with minimal supervision.

Chapter: 5

Result

Actual circuit when you implement the entire components together :

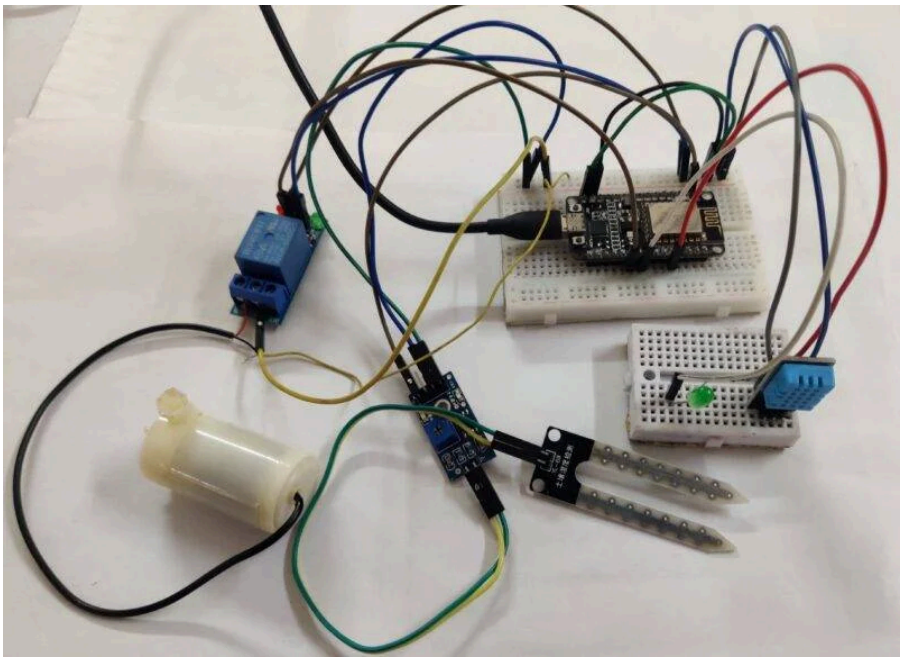


Figure 5.1: final circuit implemented

Google Sheets Reports : It is store all sensor and water pump data in sheet

IOT SENSOR DASHBOARD : Sheet1

Timestamp	Moisture	Temperature	Humidity	Pump Status		0 = PUMP OFF
Timestamp	Moisture	Temperature	Humidity	Pump Status		1 = PUMP ON
03/04/2025 05:12:52	282	0	0	1		0 = PUMP OFF
03/04/2025 17:58:30	1949	1167	2045	0		1 = PUMP OFF
03/04/2025 17:59:25	183	0	0	1		0 = PUMP OFF
03/04/2025 17:59:45	183	0	0	1		1 = PUMP OFF
03/04/2025 18:00:30	0	0	0	1		0 = PUMP OFF
03/04/2025 18:00:43	0	0	0	0		1 = PUMP OFF
03/04/2025 18:01:09	651	0	0	0		1 = PUMP ON
03/04/2025 18:02:27	1437	688	0	0		0 = PUMP OFF
03/04/2025 18:02:47	1379	585	0	0		1 = PUMP ON
03/04/2025 18:03:16	64	0	0	1		
03/04/2025 18:03:24	64	0	0	1		
03/04/2025 18:03:40	350	0	0	1		
03/04/2025 18:03:52	48	0	0	1		
03/04/2025 18:04:26	1040	127	0	0		
03/04/2025 18:04:46	612	0	0	0		
03/04/2025 18:05:00	560	249	0	0		
03/04/2025 18:05:17	693	331	0	0		
24/04/2025 19:21:52	3899	33	24	1		
24/04/2025 19:22:02	3875	33	24	1		
24/04/2025 19:22:13	3973	33	24	1		
24/04/2025 19:22:24	3959	33	24	1		
24/04/2025 19:22:35	4023	33	24	1		
24/04/2025 19:22:46	3984	33	24	1		
24/04/2025 19:22:56	4075	33	25	1		
24/04/2025 19:23:08	4095	33	25	1		
24/04/2025 19:23:18	3911	33	25	1		
24/04/2025 19:23:28	3839	33	26	1		
24/04/2025 19:23:38	3397	33	26	1		
24/04/2025 19:23:48	3928	33	26	1		

Figure 5.2: Google Sheets Data Collected from sensor

This google sheets represent that all sensor data collected on this sheet for analysis purposes also include the water pump status . Which is included in this project.

Thingspeak Cloud : It collects the data from sensors and stores it in graphical form.

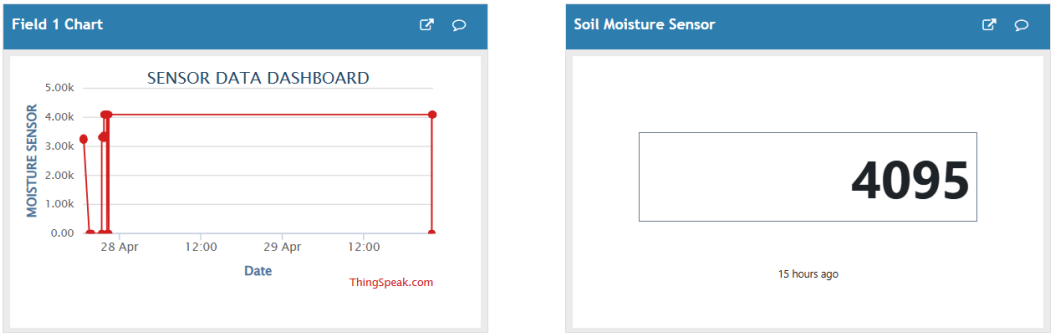


Figure 5.3: Moisture Sensor Data Collected on thingSpeak Cloud

This thingSpeak cloud graph represents soil moisture sensor data which is send by the ESP32 device for analysis purposes on the cloud .

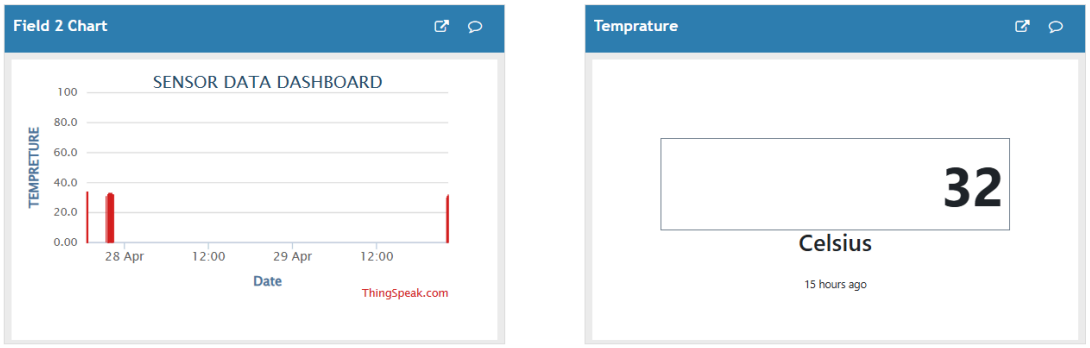


Figure 5.4: Temperature Sensor Data Collected on thingSpeak Cloud

This thingSpeak cloud graph represents temperature sensor data which is send by the ESP32 device for analysis purposes on the cloud .

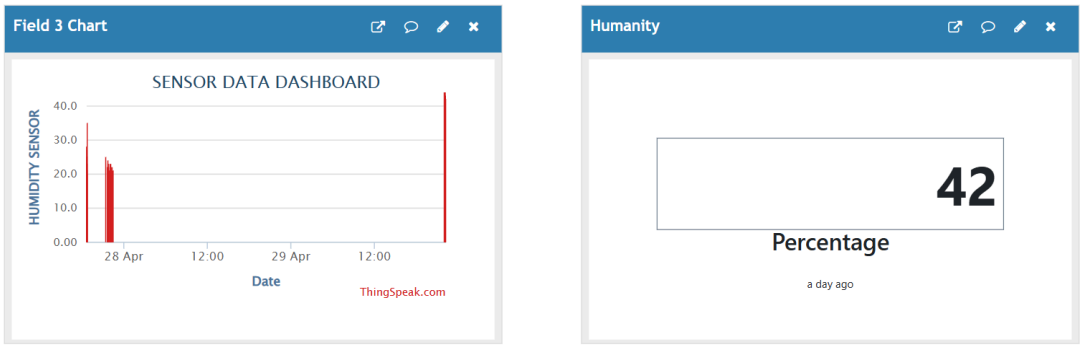


Figure 5.5: Humidity Sensor Data Collected on thingSpeak Cloud

This thingSpeak cloud graph represents humidity sensor data which is send by the ESP32 device for analysis purposes on the cloud .

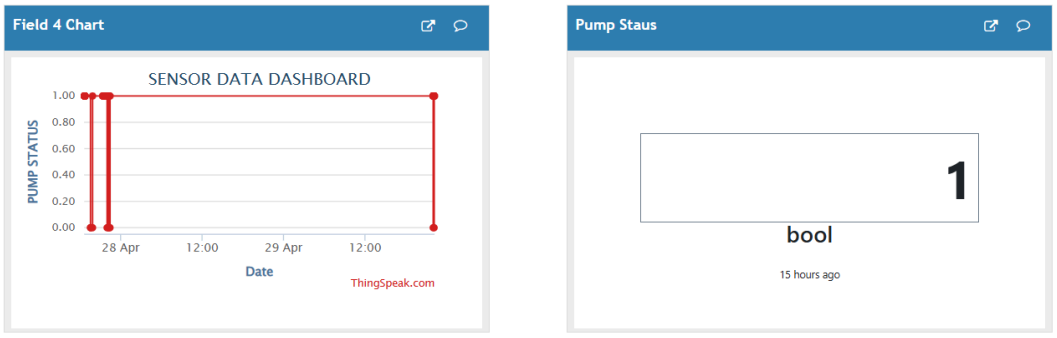


Figure 5.6: Water Pump Status Data Collected on thingSpeak Cloud

This thingSpeak cloud graph represents water pump status data which is send by the ESP32 device for analysis purposes on the cloud .

Chapter : 6

Code Explanation

In this section, we explain the code function-wise which is given below:

7.1. Overview

- Monitors soil moisture, temperature, and humidity.
- Controls a water pump based on sensor readings.
- Logs data to ThingSpeak and Google Sheets.
- Sends real-time alerts and reports via Telegram Bot.
- Supports changing Wi-Fi credentials remotely via chat.

7.2. Main Components

Libraries Used

- ➢ WiFi.h / WiFiClientSecure: Connects to the internet.
- ➢ UniversalTelegramBot.h: Handles Telegram bot interactions.
- ➢ ThingSpeak.h: For cloud data logging and timestamp.
- ➢ ArduinoJson.h: JSON parsing from APIs.
- ➢ DHT.h: Reads DHT11 temperature/humidity sensor.

7.3. Wi-Fi and Cloud Integration

Wi-Fi Credentials

```
// default ID & word always fixed
const char* ssid = "OPPO A12";
const char* password = "00000000";
```

Telegram Bot Setup

```
#define BOT_TOKEN "7122121223:AAFCCB2NjY3Z27BpPNwaM1RMFmJ1QhAiHes"
  // Telegram Bot Token
#define CHAT_ID "1079071197"
   // Admin Chat ID
```

ThingSpeak

Used to:

- ➢ Fetch timestamp
- ➢ Upload sensor data
- ➢ Generate graphical reports

```
unsigned long channelID = 2870729;
const char* apiKey = "4LDMRD21WTOE5ZIC";
```

Google Sheets

Logs events through Google Sheets.

7.4. Sensors and Actuators

- ➢ **Soil Moisture Sensor**: Analog pin 34
- ➢ **DHT11** (Temp + Humidity): Digital pin 15
- ➢ **Water Pump Relay**: Pin 5

7.5. Main Functional Features

Loop Logic

- ➢ Continuously reads sensor data
- ➢ Controls pump based on:
  - ○ dryThreshold: 2000 → turn pump ON
  - ○ wetThreshold: 700 → turn pump OFF
- ➢ Logs changes if sensor values or pump state changes
- ➢ Sends scheduled report every 12 hours

7.6. Telegram Bot Features

Supported Commands:

Command	Description
/PUMP_ON / /PUMP_OFF	Manually turn pump ON/OFF

/PUMP_STATUS	Get current pump status
/CURRENT_SENSOR_DATA	Get live sensor data
/SENSOR_STATISTICS	Shows last 10 sensor events
/TODAY_REPORT	Lists all events from today
/MONTHALY_REPORTS	Shows last 30 events
/New_WiFi_Add	Start process to change Wi-Fi
/developer_information	Show developer credits

Wi-Fi Change via Chat

- Step-by-step interaction via Telegram:
    1. Send /New_WiFi_Add
    2. Bot asks for SSID
    3. Bot asks for Password
    4. ESP32 attempts to connect and confirms success/failure

Security

➢ Only 2 authorized CHAT_IDs can control the bot.

➢ Unauthorized users receive "Access Denied" messages.

7.7. Cloud Logging & Reporting

ThingSpeak:

➢ Uploads data for real-time monitoring and graphing.

➢ Uses it to fetch current UTC timestamp and convert it to IST.

Google Sheets:

➢ Logs data via Apps Script URL

➢ Generates shareable sheets

7.9. Function-by-Function Breakdown of Your Code

void setup()

Purpose:

Initializes all components — serial, sensors, Wi-Fi, cloud services, and bot.

How it works:

- ➢ Start the serial monitor.
- ➢ Sets up pin modes (e.g., pump relay).
- ➢ Connects to Wi-Fi using stored credentials.
- ➢ Starts the DHT sensor and sets up NTP for IST time.
- ➢ Initializes the ThingSpeak and Telegram bot connections.
- ➢ Send a "Bot is online!" message to registered Telegram users

void loop()

Purpose:

Runs repeatedly to read sensors, auto-control pump, log data, and check Telegram.

How it works:

- ➢ Reads data from soil moisture sensor and DHT11.
- ➢ Controls the pump based on moisture thresholds.
- ➢ Logs data if there's a change.
- ➢ Send data to ThingSpeak and Google Sheets.
- ➢ Sends a full scheduled report every 12 hours.
- ➢ Continuously checks for incoming Telegram commands.

fetchLocalTimeFromThingSpeak()

Purpose:

Retrieves the latest UTC timestamp from ThingSpeak.

How it works:

- ➢ Sends HTTP GET to ThingSpeak.
- ➢ Extracts and returns created_at field.
- ➢ Uses convertToIST() to convert to Indian time.

convertToIST(String utcTime)

Purpose:

Converts UTC timestamps into Indian Standard Time (UTC +5:30).

How it works:

- ➢ Extracts date and time components from the UTC string.
- ➢ Adds 5 hours and 30 minutes.
- ➢ Returns formatted IST timestamp.

sendTelegramAlert(String message)

Purpose:

Sends a notification message to both registered Telegram users.

How it works:

- ➢ Uses the Telegram bot library to send a message to bot

updateThingSpeak()

Purpose:

Sends live sensor and pump data to the ThingSpeak cloud.

How it works:

- ➢ Uses ThingSpeak's setField() to assign data.
- ➢ Calls writeFields() with the API key to upload the data.

updateGoogleSheets()

Purpose:

Logs data to a Google Sheet using a web request to a Google Apps Script URL.

How it works:

- ➢ Builds a URL with moisture, temperature, humidity, and pump state.
- ➢ Sends an HTTP GET request to google Sheets endpoint.

logEvent()

Purpose:

Stores current readings and timestamp into an in-memory history array (Event struct).

How it works:

- ➢ Creates an Event with current sensor and pump status.
- ➢ Adds it to the history[] array.
- ➢ If full, shifts data and adds the newest at the end.

pumpControl(bool state, String chat_id)

Purpose:

Turn the water pump ON/OFF either automatically or through user commands.

How it works:

- ➢ Changes GPIO pin output to control the pump relay.
- ➢ Send a Telegram alert about the change.
- ➢ Logs the event and updates cloud platforms.

checkTelegramMessages()

Purpose:

Listens and responds to Telegram bot commands.

How it works:

- ➢ Verifies that the command comes from an authorized chat ID.
- ➢ Supports commands:
  - ○ /PUMP_ON, /PUMP_OFF
  - ○ /CURRENT_SENSOR_DATA
  - ○ /SENSOR_STATISTICS

- /TODAY_REPORT
- /MONTHALY_REPORTS
- /New_WiFi_Add
- /developer_information

➢ Handles Wi-Fi change interaction by storing new SSID and password.

pumpStatusReport(String chat_id)

Purpose:

Sends current pump status to the Telegram user.

How it works:

➢ Checks pumpStatus boolean.

➢ Sends ON/OFF status messages to registered users.

sendSensorData(String chat_id)

Purpose:

Sends **live sensor readings** to the user via Telegram.

How it works:

➢ Formats moisture, temperature, humidity, and pump status into a string.

➢ Send the data to the requesting Telegram chat.

sendSensorStatistics(String chat_id)

Purpose:

Shows the **last 10 events** with detailed readings and pump activity history.

How it works:

➢ Iterates through the last 10 records in history[].

➢ Shows changes in sensor values and pump states.

➢ Counts how many times the pump changed state.

sendTodayReport(String chat_id)

Purpose:

Send a detailed report of **today's events**, with a link to a chart and spreadsheet.

How it works:

- ➢ Filters events that happened today.
- ➢ Displays all events with timestamp and readings.
- ➢ Appends the ThingSpeak chart and Google Sheet report links.

sendMonthlyReport(String chat_id)

Purpose:

Sends the **last 30 events** or **full log** as a monthly report.

How it works:

- ➢ Gather up to 30 records from history[].
- ➢ Send the summary along with chart and report links.

sendScheduleReport(String chat_id)

Purpose:

Automatically sends a full report every 12 hours.

How it works:

- ➢ use a timer to send these reports.

showDeveloperNames(String chat_id)

Purpose:

Shows developer credits and project information.

.How it works:

- ➢ Send a message with names of contributors and submission information.

URLEncode(const String &str)

Purpose:

Encodes strings to be safely used in URLs (for Sheets API, etc.).

How it works:

- ➢ Replaces non-alphanumeric characters with %XX hex codes.
- ➢ Keeps regular letters/numbers unchanged.

7.10 All feature of code and its uses

In this project is a feature-rich, IoT-based smart irrigation solution. which makes it powerful:

Feature	Use
Wi-Fi control	Change via Telegram
Soil monitoring	Moisture, Temp, Humidity
Pump automation	Based on soil dryness
Telegram bot	Commands & alerts
ThingSpeak	Logs + timestamp
Google Sheets	View reports
Secure access	Only trusted chat IDs
Data logging	Up to 200 events in memory
Graph + Report	Links to charts and google sheets

7.11. Full source code of this project

See full code of this project in Appendix page or scan

scan me for full code

Chapter : 8

Application

The application of this project is the vast area of application which is given below:

Agricultural Farming

➢ Smart irrigation of large open fields (crops like wheat, rice, and vegetables).

➢ Soil moisture-driven automatic watering to avoid under- or overwatering.

➢ Real-time climate monitoring (temperature and humidity).

Advantages:

- Saves huge amounts of water .
- Increases crop yield by maintaining optimal soil moisture.
- Reduces labor cost — no need to manually monitor and irrigate.

Home Gardens and Lawns

➢ Smartly waters residential garden plants, flower beds, or home lawns.

➢ Triggers watering only when the soil becomes dry.

➢ Manual pump ON/OFF control through a simple Telegram command.

Advantages:

- Healthy plants without wasting water.
- No worries even if the homeowner is on vacation .
- Reduces the need for full-time gardeners.

Golf Courses and Parks

➢ Maintains lush green grass by intelligent irrigation scheduling.

➢ Divides large areas into zones and waters based on soil dryness.

Advantages:

- Saves huge operational water costs .
- Preserves the aesthetics and health of parks/golf lawns.
- Real-time control from maintenance offices using a phone.

Industrial Agriculture

➢ Used in large-scale farming businesses.

➢ Connects multiple fields into a central dashboard (ThingSpeak, Google Sheets).

Advantages:

- Full cloud tracking of irrigation logs.
- Scheduled reports help managers make better farming decisions.
- Centralized command over hundreds of pumps across fields.

Urban Landscaping

➢ Watering of vertical gardens, green walls, rooftop gardens in urban cities.

➢ Control watering remotely without sending physical workers daily.

Advantages:

- Adds greenery to cities while saving manpower and water.
- Greatly improves maintenance of public spaces like city squares.

Chapter : 7

Conclusion

The “Smart Irrigation System using IOT and Cloud” project successfully combines IoT, sensor automation, cloud integration, and Telegram bot control into a single, affordable, and scalable solution for modern irrigation management. By using real-time data from soil moisture, temperature, and humidity sensors, the system ensures that water is supplied only when needed — promoting sustainable water usage and improving crop health.

One of the key strengths of this project lies in its user accessibility. Through a Telegram bot interface, users can remotely monitor live sensor data, manually control the pump, change Wi-Fi settings, and receive scheduled reports — even without any technical background. The system also supports data logging to ThingSpeak and Google Sheets, enabling users to visualize performance trends and keep historical records.

This project not only demonstrates a working model of smart agriculture but also proves that IoT-based automation can be made simple, practical, and impactful, addressing critical issues in water management and labor efficiency in agriculture.

1. Full source code of Smart irrigation System Using IOT and Cloud

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <UniversalTelegramBot.h>
#include <ThingSpeak.h>
#include <ArduinoJson.h>
#include "DHT.h"
// ==========================
// WiFi Credentials
// ==========================
const char* ssid = "OPPO A12";
const char* password = "00000000";
// === Flags ===
bool awaitingSSID = false;
bool awaitingPASS = false;
String newSSID = "";
String newPASS = "";
// ==========================
// Telegram Credentials
// ==========================
#define BOT_TOKEN
"7122121223:AAFCCB2NjY3Z27BpPNwaM1RMFmJ1QhAiHes"
#define CHAT_ID  "1079071197"
#define CHAT_ID_yash  "6741824552"
```

```
#define chat_id_yash "6741824552"

// Secure client for Telegram

WiFiClientSecure telegramClient;

UniversalTelegramBot bot(BOT_TOKEN, telegramClient);

// ============================

// ThingSpeak Credentials

// ============================

unsigned long channelID = 2870729;     // Replace with your ThingSpeak channel ID

const char* apiKey = "4LDMRD21WTOE5ZIC"; // Replace with your ThingSpeak Write API key

WiFiClient thingSpeakClient;

 // Used for ThingSpeak API calls

// ============================

// Google Sheets Logging URL

// ============================

const char* sheetsURL = "https://script.google.com/macros/s/AKfycbzdEdhkiBYyXYI7l-9ynTfivWdu4JxrpO_PX6VFVzagFU75zdBtnmcaC2scgWjdlR10/exec";

// ============================

// Sensor & Pump Pins

// ============================

#define MOISTURE_SENSOR_PIN 34

#define DHTPIN 15          // Pin connected to DHT11 data

#define DHTTYPE DHT11      // DHT 11 sensor

#define PUMP_PIN 5

DHT dht(DHTPIN, DHTTYPE);

// ============================

// Global Variables: Sensor Readings, Pump State & Thresholds
```

```
// ==========================
int moistureLevel = 0;
int temperature = 0;
int humidity = 0;
bool pumpStatus = false;  // false = OFF; true = ON
// Moisture threshold values for automatic pump control
int dryThreshold = 2000; // Pump should turn ON if moisture < dryThreshold (dry soil)
int wetThreshold = 700; // Pump should turn OFF if moisture > wetThreshold (wet soil)
// ------------------------
// Event Logging Structures
// ------------------------
#define MAX_HISTORY 200
struct Event {
  String timestamp;   // Fetched from ThingSpeak
  int moisture;
  int temperature;
  int humidity;
  bool pumpStatus;
};
Event history[MAX_HISTORY];
int historyCount = 0;  // Number of events stored
// ------------------------
// Previous Values for Change Detection
// ------------------------
int prevMoisture    = -1;
int prevTemperature = -1;
int prevHumidity    = -1;
```

```
bool prevPumpStatus = false;
// ------------------------
// Timing Variables
// ------------------------
unsigned long lastReportTime = 0;
unsigned long lastTelegramCheck = 0;
const unsigned long reportInterval = 43200000; // 12 hours (in ms)
const unsigned long telegramCheckInterval = 5;   // Check Telegram every 5 ms
// ==========================
// Function Prototypes
// ==========================
String fetchLocalTimeFromThingSpeak();          // Fetches real-time timestamp from ThingSpeak server
void sendTelegramAlert(String message);
void pumpControl(bool state, String chat_id);
void pumpStatusReport(String chat_id);
void sendSensorData(String chat_id);
void sendSensorStatistics(String chat_id);
void sendTodayReport(String chat_id);
void sendMonthlyReport(String chat_id);
void sendScheduleReport(String chat_id); // Full event history
void checkTelegramMessages();
void updateThingSpeak();
void updateGoogleSheets();
void logEvent();   // Log event based on current sensor & pump state
void sendTelegramAlert();
String convertToIST(String utcTime);
void showDeveloperNames(String chat_id);
```

```
String URLEncode(const String &str);
// ============================
// Setup
// ============================
 void setup() {
 Serial.begin(115200);
 dht.begin();
 pinMode(PUMP_PIN, OUTPUT);
 digitalWrite(PUMP_PIN, LOW);
 Serial.print("Connecting to WiFi");
 WiFi.mode(WIFI_STA);
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
  delay(200);
  Serial.print(".");
 }
 Serial.println("\nWiFi connected!");
 // Set up TLS/SSL certificate for Telegram connections
telegramClient.setCACert(TELEGRAM_CERTIFICATE_ROOT);
// **Initialize Telegram Client**
  telegramClient.setInsecure();
   // **Connect to ThingSpeak**
 ThingSpeak.begin(thingSpeakClient);
 // Initialize NTP (set to IST: UTC+5:30)
 configTime(19800, 0, "pool.ntp.org");
// Send message
 bot.sendMessage(CHAT_ID, "🌱 ESP32 Connected to Telegram :Bot is online!", "");
```

```
bot.sendMessage(CHAT_ID_yash, "🌱 ESP32 Connected to Telegram :Bot is online!",
"");
  if (bot.sendMessage(CHAT_ID, "🌱 ESP32 Connected to Telegram :Bot is online!",
"")) {
    Serial.println("Startup message sent.");
  } else {
    Serial.println("Failed to send startup message.");
  }
}
// ==========================
// Main Loop
// ==========================
 void loop() {
  // Read sensor values from analog pins
  int newMoisture    = analogRead(MOISTURE_SENSOR_PIN);
  int newTemperature = dht.readTemperature();  // Celsius
  int newHumidity    = dht.readHumidity();
checkTelegramMessages();
  moistureLevel = newMoisture;
  temperature   = newTemperature;
  humidity      = newHumidity;
// Debug output - print current sensor readings, pump state and timestamp
  Serial.println("----- SENSOR UPDATE -----");
  Serial.println("Moisture:    " + String(moistureLevel));
  Serial.println("Temperature: " + String(temperature));
  Serial.println("Humidity:    " + String(humidity));
  Serial.println("Pump Status: " + String(pumpStatus ? "ON" : "OFF"));
  Serial.println("Timestamp:   " + fetchLocalTimeFromThingSpeak());
```

```
  Serial.println("-------------------------");
// Automatic pump control based on moisture thresholds
  if (moistureLevel > dryThreshold && pumpStatus == false) {
    pumpControl(true, CHAT_ID);
  } else if (moistureLevel < wetThreshold && pumpStatus == true) {
    pumpControl(false, CHAT_ID);
  }
checkTelegramMessages();
  // If any sensor value or pump state changed, log event and update cloud immediately
  if (newMoisture != prevMoisture || newTemperature != prevTemperature ||pumpStatus != prevPumpStatus) {
    logEvent();
    updateThingSpeak();
    updateGoogleSheets();
    prevMoisture = newMoisture;
    prevTemperature = newTemperature;
    prevHumidity = newHumidity;
    prevPumpStatus = pumpStatus;
  }
  checkTelegramMessages();
  // Scheduled full history report (every 12 hours)
  if (millis() - lastReportTime > reportInterval) {
    sendScheduleReport(CHAT_ID);
    lastReportTime = millis();
  }
  }
// ==========================
// Fetch Local Time
```

```
// ==========================
 String fetchLocalTimeFromThingSpeak() {
  HTTPClient http;
  String url = String("https://api.thingspeak.com/channels/") + String(channelID) +
"/feeds.json?results=1";
  http.begin(url);
  int httpCode = http.GET();
  String localTime = "Error";
  if (httpCode == HTTP_CODE_OK) {
    String payload = http.getString();
    DynamicJsonDocument doc(1024);
    DeserializationError error = deserializeJson(doc, payload);
    if (!error) {
      String utcTime = doc["feeds"][0]["created_at"].as<String>();
      localTime = convertToIST(utcTime);  // Convert UTC to IST
    }
  }
  http.end();
  return localTime;
}
String convertToIST(String utcTime) {
  int year = utcTime.substring(0, 4).toInt();
  int month = utcTime.substring(5, 7).toInt();
  int day = utcTime.substring(8, 10).toInt();
  int hour = utcTime.substring(11, 13).toInt();
  int minute = utcTime.substring(14, 16).toInt();
  int second = utcTime.substring(17, 19).toInt();
 // Adjust UTC to IST (+5:30)
```

```
  hour += 5;
  minute += 30;
  if (minute >= 60) {
    minute -= 60;
    hour += 1;
  }
  if (hour >= 24) {
    hour -= 24;
    day += 1;
  }
  char formattedTime[20];
  sprintf(formattedTime, "%02d/%02d/%04d %02d:%02d:%02d", day, month, year, hour,
minute, second);
  return String(formattedTime);
}
 void sendTelegramAlert(String message) {
  bot.sendMessage(CHAT_ID, message, "");
  bot.sendMessage(CHAT_ID_yash, message, "");
 }
  void updateThingSpeak() {
  ThingSpeak.setField(1, moistureLevel);
  ThingSpeak.setField(2, temperature);
  ThingSpeak.setField(3, humidity);
  ThingSpeak.setField(4, pumpStatus ? 1 : 0);
  int response = ThingSpeak.writeFields(channelID, apiKey);
  if (response == 200) {
    Serial.println("✅ Data sent to ThingSpeak successfully!");
  } else {
```

```
    Serial.println("❌ ThingSpeak Update Failed. Error Code: " + String(response));
  }
}
void updateGoogleSheets() {
  HTTPClient http;
  String url = String(sheetsURL) +
               "?moisture="    + String(moistureLevel) +
               "&temperature=" + String(temperature) +
               "&humidity="    + String(humidity) +
               "&pump="        + String(pumpStatus ? 1 : 0);
  Serial.println("Google Sheets URL: " + url);
  http.begin(url);
  int httpCode = http.GET();

  if (httpCode > 0) {
    Serial.println("✅ Google Sheets Update Sent. Response code: " + String(httpCode));
    String response = http.getString();
    Serial.println("Response: " + response);
  } else {
    Serial.println("❌ Google Sheets Update Failed. Error: " +
http.errorToString(httpCode));
  }
  http.end();
}
// ============================
// Event & Serial Handling
// ============================
// Log an event with the current sensor readings, pump state, and timestamp
```

```
void logEvent() {
  Event ev;
  ev.timestamp = fetchLocalTimeFromThingSpeak();
  ev.moisture = moistureLevel;
  ev.temperature = temperature;
  ev.humidity = humidity;
  ev.pumpStatus = pumpStatus;

  if (historyCount < MAX_HISTORY) {
    history[historyCount++] = ev;
  } else {
    // Shift left if history full, then append the new event
    for (int i = 1; i < MAX_HISTORY; i++) {
      history[i - 1] = history[i];
    }
    history[MAX_HISTORY - 1] = ev;
  }
  Serial.println("Event logged: " + ev.timestamp);
}
// ============================A
// Pump Control & Reporting via Telegram Commands
// ============================
void pumpControl(bool state, String chat_id) {
  pumpStatus = state;
  digitalWrite(PUMP_PIN, state ? HIGH : LOW);
  String msg = state ? "🚨 PUMP TURN ON 🚨\n" : "Pump turned OFF.\n";
  msg += "🌱 Moisture: " + String(moistureLevel) + "\n";
```

```
  msg += " 🌡️ Temperature: " + String(temperature) + "\n";
  msg += " 💧 Humidity:" + String(humidity) + "\n";
  msg += "Timestamp: " + fetchLocalTimeFromThingSpeak();
  bot.sendMessage(chat_id, msg, "");
  bot.sendMessage(chat_id_yash, msg, "");
  updateThingSpeak();
  updateGoogleSheets();
  logEvent();
  prevPumpStatus = pumpStatus;
}
void pumpStatusReport(String chat_id) {
  String status = pumpStatus ? "ON" : "OFF";
  bot.sendMessage(chat_id, "Current pump status: " + status, "");
  bot.sendMessage(chat_id_yash, "Current pump status: " + status, "");
}
void sendSensorData(String chat_id) {
  String msg = String("🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
            "Current Sensor Data:\n") +
             "🌱 Soil Moisture:  " + String(moistureLevel) + "\n" +
             " 🌡️ Temperature: " + String(temperature) + "\n" +
             " 💧 Humidity: " + String(humidity) + "\n" +
             "🚰 Pump: " + (pumpStatus ? "ON" : "OFF") + "\n" +
             "Timestamp: " + fetchLocalTimeFromThingSpeak();
  bot.sendMessage(chat_id, msg, "");
  bot.sendMessage(chat_id_yash, msg, "");
}
void sendSensorStatistics(String chat_id) {
```

```
  int count = (historyCount < 10) ? historyCount : 10;
  String msg = "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
   "Last " + String(count) + " events:\n";
  int pumpChangeCount = 0;
  for (int i = historyCount - count; i < historyCount; i++) {
   msg += "[" + history[i].timestamp + "] \n🌱 Moisture:" + String(history[i].moisture)
+"\n" +
          " 🌡️ Temperature:" + String(history[i].temperature) +"\n" +
          " 💧 Humidity:" + String(history[i].humidity) +"\n" +
          " 🚰 Pump:" + (history[i].pumpStatus ? "ON" : "OFF") + "\n";
   if (i > historyCount - count && history[i].pumpStatus != history[i - 1].pumpStatus) {
    pumpChangeCount++;
   }
  }
  msg += "Pump state changed " + String(pumpChangeCount) + " times in these events.";
  bot.sendMessage(chat_id, msg, "");
  bot.sendMessage(chat_id_yash, msg, "");
}
void sendTodayReport(String chat_id) {
  String currentTime = fetchLocalTimeFromThingSpeak();
  String today = currentTime.substring(0, 10);
  String msg = "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
   "Today's Events (" + today + "):\n";
  for (int i = 0; i < historyCount; i++) {
   if (history[i].timestamp.startsWith(today)) {
    msg += "[" + history[i].timestamp + "] \n🌱 Soil Moisture:" +
String(history[i].moisture) +"\n" +
          " 🌡️ Temperature:" + String(history[i].temperature) +"\n" +
```

```
      " 💧 Humidity: " + String(history[i].humidity) +"\n" +
      " 🚰 Pump:" + (history[i].pumpStatus ? "ON" : "OFF") + "\n";
   }
  }
  msg += "\n🔗 Graph : https://thingspeak.com/channels/" + String(channelID) +
"/charts";
  msg += "\n📊 Report Sheet:
https://docs.google.com/spreadsheets/d/e/2PACX-1vQEFRp4uLweyKxa7MrYbVdbuLC_
Dm-9AnYlwRqPDxnfg5vW8cKm9o-BjSC6Y0rO5-v4PzF6vC4SBibh/pubhtml";
  bot.sendMessage(chat_id, msg, "");
  bot.sendMessage(chat_id_yash, msg, "");
}
void sendMonthlyReport(String chat_id) {
  int count = (historyCount < 200) ? historyCount : 30;
  String msg = "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
  "Last " + String(count) + " events (Monthly Stats):\n";
  for (int i = historyCount - count; i < historyCount; i++) {
   msg += "[" + history[i].timestamp + "] \n🌱 Soil Moisture:" +
String(history[i].moisture) +"\n" +
      " 🌡️ Temperature:" + String(history[i].temperature) +"\n" +
      " 💧 Humidity: " + String(history[i].humidity) +"\n" +
      " 🚰 Pump:" + (history[i].pumpStatus ? "ON" : "OFF") + "\n";
  }
  msg += "🔗 Graph : https://thingspeak.com/channels/" + String(channelID) + "/charts";
   msg += "\n📊 Report Sheet:
https://docs.google.com/spreadsheets/d/e/2PACX-1vQEFRp4uLweyKxa7MrYbVdbuLC_
Dm-9AnYlwRqPDxnfg5vW8cKm9o-BjSC6Y0rO5-v4PzF6vC4SBibh/pubhtml";
  bot.sendMessage(chat_id, msg, "");
   bot.sendMessage(chat_id_yash, msg, "");
```

```
}
void sendScheduleReport(String chat_id) {
  String msg =  "📊 SCHEDULED REPORT 📊\n"
                "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n";
  for (int i = 0; i < historyCount; i++) {
    msg += "[" + history[i].timestamp + "] \n🌱 Soil Moisture:" +
String(history[i].moisture) +"\n" +
          " 🌡️ Temperature:" + String(history[i].temperature) +"\n" +
          " 💧 Humidity: " + String(history[i].humidity) +"\n" +
          " 🚰 Pump:" + (history[i].pumpStatus ? "ON" : "OFF") + "\n";
  }
  msg += "🔗 Graph: https://thingspeak.com/channels/" + String(channelID) + "/charts";
   msg += "\n📊 Report Sheet:
https://docs.google.com/spreadsheets/d/e/2PACX-1vQEFRp4uLweyKxa7MrYbVdbuLC_
Dm-9AnYlwRqPDxnfg5vW8cKm9o-BjSC6Y0rO5-v4PzF6vC4SBibh/pubhtml";
  bot.sendMessage(chat_id, msg, "");
  bot.sendMessage(chat_id_yash, msg, "");
}
// ============================
// Telegram Command Handling
// ============================
void checkTelegramMessages() {
  int numNewMessages = bot.getUpdates(bot.last_message_received + 1);
  while (numNewMessages) {
    for (int i = 0; i < numNewMessages; i++) {
      String chat_id = bot.messages[i].chat_id;
      String text    = bot.messages[i].text;
      Serial.printf("Received from %s: %s\n", chat_id.c_str(), text.c_str());
```

```
    if (chat_id != CHAT_ID && chat_id != chat_id_yash) {          // version feature add
yash id access
      bot.sendMessage(chat_id, "⛔ Access Denied : Unauthorized Access!");
      return;
    }   else if (text == "/New_WiFi_Add") {
      bot.sendMessage(chat_id, "📶 Send new Wi-Fi SSID:");
      awaitingSSID = true;
      awaitingPASS = false;
    }
    else if (awaitingSSID) {
      newSSID = text;
      bot.sendMessage(chat_id, " 🔒 Now send Wi-Fi Password:");
      awaitingSSID = false;
      awaitingPASS = true;
    }
    else if (awaitingPASS) {
      newPASS = text;
      bot.sendMessage(chat_id, "🔄 Trying to connect to: " + newSSID);
      awaitingPASS = false;
      // Try new Wi-Fi without disconnecting first
      WiFi.begin(newSSID.c_str(), newPASS.c_str());
      int retry = 0;
      while (WiFi.status() != WL_CONNECTED && retry < 10) {
        delay(1000);
        retry++; }
      if (WiFi.status() == WL_CONNECTED) {
```

```
      // Now disconnect previous and reconnect cleanly
      WiFi.disconnect(true); // Disconnect from current
      delay(500);
      WiFi.begin(newSSID.c_str(), newPASS.c_str());
      retry = 0;
      while (WiFi.status() != WL_CONNECTED && retry < 10) {
        delay(1000);
        retry++;
      }
      if (WiFi.status() == WL_CONNECTED) {
        bot.sendMessage(chat_id, "✅ Connected to " + newSSID + "\n🌐 IP: " +
WiFi.localIP().toString());
      } else {
        bot.sendMessage(chat_id, "❌ Failed to reconnect after disconnect.");
      }
    } else {
      bot.sendMessage(chat_id, "❌ Failed to connect. Please check SSID/Password.");
    }
  }
  else if (text == "/PUMP_ON") {
    pumpControl(true, chat_id);
  } else if (text == "/PUMP_OFF") {
    pumpControl(false, chat_id);
  } else if (text == "/PUMP_STATUS") {
    pumpStatusReport(chat_id);
  } else if (text == "/CURRENT_SENSOR_DATA") {
    sendSensorData(chat_id);
  } else if (text == "/SENSOR_STATISTICS") {
```

```
      sendSensorStatistics(chat_id);
    } else if (text == "/TODAY_REPORT") {
      sendTodayReport(chat_id);
    } else if (text == "/MONTHALY_REPORTS") {
      sendMonthlyReport(chat_id);
    }else if (text == "/developer_information") {
    showDeveloperNames(chat_id);
   }
else {
      bot.sendMessage(chat_id, "Hi I Am Bot For Your Help\n"
                  "Your Available Commands Here :\n"
                  "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
                  "                 ▮▮▮▮▮▮    \n"
                  "🌱🌿 🫧/PUMP_ON,🫧 🌿🌱 \n"
                  "                 ▮▮▮▮▮▮    \n"
                  "🌱🌿 🫧/PUMP_OFF,🫧 🌿🌱\n"
                  "                 ▮▮▮▮▮▮    \n"
                  "🌱🌿 🫧/PUMP_STATUS,🫧 🌿🌱\n"
                  "                 ▮▮▮▮▮▮     \n"
                  "🌱🌿/CURRENT_SENSOR_DATA,🌿🌱\n"
                  "                 ▮▮▮▮▮▮     \n"
                  "🌱🌿 /SENSOR_STATISTICS, 🌿🌱\n"
                  "                 ▮▮▮▮▮▮     \n"
                  "🌱🌿 🫧/TODAY_REPORT,🫧 🌿🌱\n"
                  "                 ▮▮▮▮▮▮     \n"
                  "🌱🌿🫧/MONTHALY_REPORTS,🫧🌿🌱\n"
                  "                 ▮▮▮▮▮▮     \n"
```

```
                "🌱🌿🫧 /New_WiFi_Add -Change Wi-Fi via chat🫧🌿🌱\n"
                "                ▮▮▮▮▮▮▮      \n"
                " 🫧🌱🌿🫧  Please Wait 🫧 🌱🌿🫧\n"
                " 🌱🫧  Till Update The Bot Status 🫧🌱\n"
                "                ▮▮▮▮▮▮▮      \n"
                "                                \n"
                "       ❣️ 😊 Thank You 😊❣️      \n"
                "                                \n"
                " 🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
                "    /developer_information, '', "");
    }
   }
   numNewMessages = bot.getUpdates(bot.last_message_received + 1);
  }
}
void showDeveloperNames(String chat_id) {
 String developerNames = "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n"
                         "                    \n"
                         " 🫧This Project Is Made by:🫧 \n"
                         "                        \n"
                         "🫧🌿 Pawan Kumar Maurya  \n"
                         "🫧🌿 Himanshu Chaudhary  \n"
                         "🫧🌿 Yash Kumar          \n"
                         "                    \n  "
                         " Submitted To : Electronics & Communication Engineering\n"
                         " Department SCRIET CCS University\n "
                         "🌱🌼🌿 🫧🌿🫧🌿🌼🌱\n";
```

```
  bot.sendMessage(chat_id, developerNames, "");
}
String URLEncode(const String &str) {
  String encodedString = "";
  char c;
  char code0, code1;
  for (int i = 0; i < str.length(); i++) {
    c = str.charAt(i);
    if (isalnum(c)) {
      encodedString += c;
    } else {
      code1 = (c & 0xf) + '0';
      if ((c & 0xf) > 9) {
        code1 = (c & 0xf) - 10 + 'A';
      }
      c = (c >> 4) & 0xf;
      code0 = c + '0';
      if (c > 9) {
        code0 = c - 10 + 'A';
      }
      encodedString += '%';
      encodedString += code0;
      encodedString += code1;
    }
  }
  return encodedString;
}
```

full code link:-

https://github.com/pawan0011/final_code_of_smart_irrigation_system/blob/main/final_code_of_smart_irrigation_system

scan me for full code

Scan me for Quick Guide

2. Visualization

ThingSpeak Chart:

https://thingspeak.com/channels/2870729/charts

Google Sheet Report:

https://docs.google.com/spreadsheets/d/e/2PACX-1vQEFRp4uLweyKxa7MrYbVdbuLC_Dm-9AnYlwRqPDxnfg5vW8cKm9o-BjSC6Y0rO5-v4PzF6vC4SBibh/pubhtml

---

3. Quick User Guide

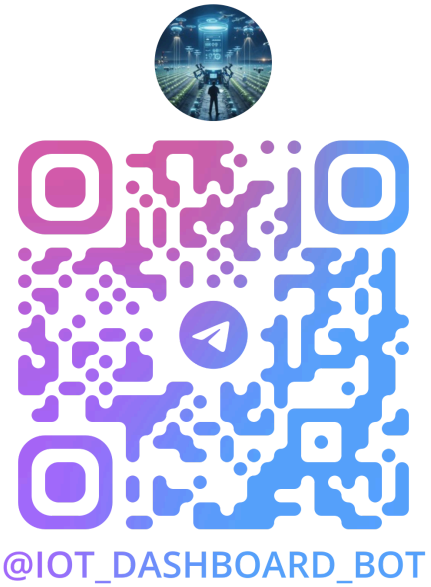
Step 1: Connect to Power

- Plug your ESP32 system into a **USB power adapter** or **power bank**.
- The device will **automatically start** and create a **Wi-Fi hotspot**.

Step 2: Connect to Default Wi-Fi

- Go to **Wi-Fi settings** on your phone or laptop.
- Connect to:
    - **Wi-Fi Name (SSID):** OPPO A12
    - **Password:** 00000000
- You are now connected directly to the ESP32 device.

Note: This Wi-Fi is just for initial setup.



Step 3: Connect to the Telegram Bot

- Open your **Telegram app**.
- **Scan the QR code below** or **search** for **@IOT_DASHBOARD_BOT** manually.

QR Code to join:

- **Bot Link : - https://t.me/IOT_DASHBOARD_BOT**

Step 4: Set Up Your Wi-Fi via Telegram

Inside the bot:

1. Type **/New_WiFi_Add** and send it.
2. Bot will ask for your **Wi-Fi name (SSID)** — reply with your Wi-Fi name.
3. Bot will then ask for your **Wi-Fi password** — reply with your password.
4. System connects to your new Wi-Fi and confirms success.

Once you see the success message, you are fully connected!

Important: If ESP32 resets or loses power, it will **revert to default Wi-Fi (OPPO A12)**.
Reconnect using steps above if needed.

Step 5: Use System Commands

After setup, test and control the system with these Telegram commands:

Telegram Command	Function
/PUMP_ON	Manually turn ON pump

/PUMP_OFF	Manually turn OFF pump
/PUMP_STATUS	Check current pump status
/CURRENT_SENSOR_DATA	Get latest Moisture, Temp, Humidity
/SENSOR_STATISTICS	See last 10 events summary
/TODAY_REPORT	Full report of today's activity
/MONTHALY_REPORTS	Report of last 30 events
/developer_information	Project and developer details

Reports are also **automatically** sent every **12 hours**.

Pump control is **automatic** based on soil moisture.

Access Control & Security

- Only **authorized users** (pre-registered Telegram accounts) can control the system.
- If someone unauthorized tries to use commands, they will receive **Access Denied**.

If you are blocked or need help, **contact the developer**.

---

4. QR code for quick guide for this project :

Scan me for Quick Guide

1. Reference

1. Schwartz, M. (2017). Internet of Things with ESP8266: Build connected IoT devices with Arduino and ESP8266. Birmingham, UK: Packt Publishing. ISBN: 9781787288106

2. Pfister, C. (2011). Getting Started with the Internet of Things: Connecting Sensors and Microcontrollers to the Cloud. Sebastopol, CA: O'Reilly Media. ISBN: 9781449393571

3. Singh, N. K. (2020). ESP32 Development using Arduino IDE: Step-by-step guide for IoT applications. Independently published. ISBN: 9798613770131

4. Rao, N. R., & Geethanjali, M. (2021). IoT and Smart Agriculture: Technology, Adoption and Prospects. Boca Raton, FL: CRC Press. ISBN: 9781774637547

5. Kumar, G. M., & Kumar, A. V. (2023). Smart Agriculture: An Approach Towards Better Agriculture Management. Hoboken, NJ: Wiley-Scrivener. ISBN: 9781119791792

2. Online resource

1. **https://randomnerdtutorials.com/esp32-pinout-reference-gpios/**
2. **https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html#**
3. **https://randomnerdtutorials.com/telegram-control-esp32-esp8266-nodemcu-outputs/**
4. **https://lastminuteengineers.com/soil-moisture-sensor-arduino-tutorial/**
5. **https://lastminuteengineers.com/soil-moisture-sensor-arduino-tutorial/**
6. **https://projecthub.arduino.cc/arcaegecengiz/using-dht11-12f621**
7. **https://randomnerdtutorials.com/esp32-thingspeak-publish-arduino/**
8. **https://thingspeak.mathworks.com/**
9. **https://medium.com/@amnahhmohammed/sensor-data-logging-with-esp32-thingspeak-6d995e756f28**
10. **https://randomnerdtutorials.com/esp32-datalogging-google-sheets/**